

Artificial Intelligence in the Knowledge Economy

Autonomy, capability, and a Lean verification exercise

Enrique Ide Eduard Talamàs (2025)

github.com/gsaco/ai-05-ide

Question and source

Who gains when knowledge becomes scalable?

We track bottom wages, top wages, and total output.

- Course reading: May 20, 2025, 39 pages.
- Lean source: arXiv v11, February 25, 2025, 35 pages.
- Focus: Propositions 5 and 6, then a three-type example.

Autonomy and capability are different dimensions

Autonomy: permitted roles

Autonomous AI can produce, work, and advise.

Co-pilot-only AI can advise.

Capability: knowledge

$$a = z_{AI} \in [0, 1).$$

The fraction of problems AI can solve.

Holding autonomy fixed does not fix who gains.

The agent's problem: organize knowledge and time

A worker with knowledge z asks a solver $s > z$ for help.

$$h n(z)(1 - z) = 1 \implies n(z) = \frac{1}{h(1 - z)}.$$

Firm profit

$$\Pi_{HH} = n(z)[s - w(z)] - w(s).$$

Humans maximize income; firms choose teams to maximize profit.

Free entry and market clearing determine wages and AI rents.

Conditions: people, problems, and firms

- Unit human mass; one unit of time per human.
- Observable $z \in [0, 1]$ with continuous, positive density.
- Independent uniform problem difficulty; risk neutrality.
- Competition, complete information, output price one.
- No fixed costs; at most two organizational layers.

Conditions: scarce time and abundant compute

- $0 < h < h_0$: no independent humans before AI.
- $a < 1$: AI is not perfectly knowledgeable.
- Production opportunities exceed production capacity.
- Compare autonomy at the same capability a and compute μ .

A sufficient compute-abundance condition

$$\mu > \int_0^a h(1-z) dG(z) + \frac{1-G(a)}{h(1-a)}.$$

Proposition 5: capability determines bottom gains

Let B and T be strict wage winners below and above AI.

Bottom: a capability threshold

$$B \neq \emptyset \iff a > \bar{a}, \quad \bar{a} \in \text{int } W.$$

W is the pre-AI worker set.

- Top: $T \neq \emptyset$ for every $a < 1$, under the stated conditions.
- Winners occupy endpoint intervals; the human with $z = a$ loses.

Proposition 6: adoption still requires capability

Co-pilot-only equilibrium has zero AI rent: $r^N = 0$.

Capability	Outcome
$a \leq w(0)$	No adoption; wages and occupations unchanged.
$a > w(0)$	Lowest-knowledge humans adopt; strict bottom gains and some strict human losses.

The adoption threshold $w(0)$ differs from the P5 threshold $\bar{a}$.

Proposition 6: bottom, top, and total output

- Near the bottom: $w^N \geq \max\{w, w^A\}$.
Strict if $a > w(0)$.
- Near the top: $w^N \leq w^A$.
Strict below $z = 1$.
- Total output: $Y^A > Y^N$.

The co-pilot equilibrium is unique, efficient, and maximizes labor income *within its permitted technology*.

A three-type economy we can solve directly

Type	z	Mass
Low	0	.55
Middle	$1/2$	.30
High	1	.15

$$h = \frac{1}{2}, \quad \mu = 2.$$

Before AI, zero profit gives

$$u + \frac{1}{2}v = \frac{1}{2},$$

$$u + \frac{1}{2}t = 1,$$

$$v + \frac{1}{4}t = 1.$$

Hence $(u, v, t) = (.2, .6, 1.6)$.

This finite example changes the paper's population assumption.

Autonomous AI: solve one equilibrium branch

For $0 < a < 1/3$, spare compute pins the rent at $r^A = a$.

Human role	Wage
High: supervise AI workers	$t = 2$
Middle: supervise AI workers	$v = (1 - 2a)/(1 - a)$
Low: work with middle solvers	$u = \frac{1}{2} - \frac{1}{2}v = \frac{a}{2(1 - a)}$

Resource clearing and no profitable entry validate this branch.

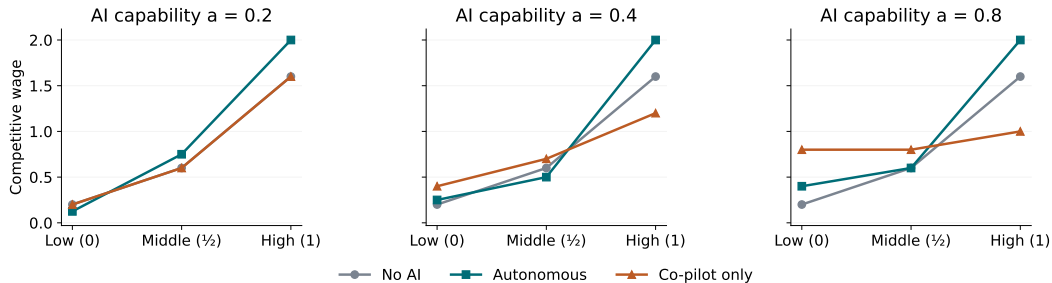
Same autonomy, different bottom outcomes

$$\frac{a}{2(1-a)} > \frac{1}{5} \iff 5a > 2(1-a) \iff a > \frac{2}{7}.$$

Capability	Bottom wage	Versus .20
$a = .20$	.125	Loss
$a = .30$	3/14	Gain

The denominator is positive on $0 < a < 1/3$. Equality is not a gain.

Numerical results separate the two dimensions



Across panels: change capability.

Within each panel: change autonomy.

More output does not imply more labor income

Same capability $a = .4$ and compute supply $\mu = 2$.

	Labor income	Compute income	Output
No AI	.5300	0	.5300
Co-pilot only	.6100	0	.6100
Autonomous	.5875	.8000	1.3875

The numerical program checks feasibility, no profitable entry, primal–dual equality, and wage uniqueness.

The claim I would not accept without checking

The misleading gloss

“Distribution depends on autonomy, not capability.”

$$\frac{a}{2(1-a)} > \frac{1}{5} \iff a > \frac{2}{7}, \quad 0 < a < \frac{1}{3}.$$

The genuine handwritten photograph is still pending.

Verdict: capability changes the sign even at fixed autonomy.

What the paper and the example teach us

- **Bottom:** capability must cross a threshold.
- **Top:** autonomy scales expertise under the stated conditions.
- **Output:** autonomous AI permits additional productive uses.

Next: what can Lean verify?

Six slides on statements, proofs, assumptions, and remaining gaps.

Lean 1/6: from economics to a precise statement

Paper claim $\longrightarrow$ Lean statement $\longrightarrow$ Proof

- A Spec : Prop states objects, assumptions, and conclusion.
- Lean checks a proof of that exact formal statement.
- Matching the statement to the paper requires a separate audit.

Our run checks supporting algebra and a finite-model threshold.

Lean 2/6: the bottom-gain threshold

$$0 < a < \frac{1}{3} \implies \left(\frac{1}{5} < \frac{a}{2(1-a)} \iff \frac{2}{7} < a \right).$$

```
def BottomGainSpec : Prop :=  
  ∀ a : ℝ, 0 < a → a < 1 / 3 →  
    (1 / 5 < bottomWage a ↔ 2 / 7 < a)  
  
theorem bottomGain : BottomGainSpec := by  
  intro a ha hupper  
  have hden : 0 < 2 * (1 - a) := by linarith  
  unfold bottomWage  
  rw [lt_div_iff0 hden]  
  constructor <;> intro h <;> linarith
```

bottomGain proves both directions over real numbers.

Build: PASS. This is the finite wage inequality, not Proposition 5.

Lean 3/6: why the inequality proof works

```
intro a ha hupper
have hden : 0 < 2 * (1 - a) := by linarith
unfold bottomWage
rw [lt_div_iff0 hden]
constructor <;> intro h <;> linarith
```

- hden: prove $2(1 - a) > 0$ from the stated domain.
- lt_div_iff₀: clear division without reversing the inequality.
- constructor: split “if and only if” into two directions.
linarith closes each direction by linear arithmetic.

Lean 4/6: zero profit implies a wage identity

$$n(z) = \frac{1}{h(1-z)}, \quad n(z)(a-w) - r = 0 \implies w = a - h(1-z)r.$$

```
have hhne : h ≠ 0 := ne_of_gt hh
have hzne : 1 - z ≠ 0 := ne_of_gt (sub_pos.mpr hz)
unfold topAutomatedProfit span at hprofit
field_simp [hhne, hzne] at hprofit
nlinarith
```

zeroProfitWage uses $h > 0$ and $z < 1$ to justify division.

Build: PASS. It checks conditional algebra over real scalars;
it does not establish market clearing or equilibrium rents.

Lean 5/6: a missing assumption changes the claim

Dropping $z < 1$ allows

$$h = \frac{1}{2}, \quad z = 1, \quad a = \frac{1}{2}, \quad w = 1, \quad r = 0.$$

- Lean's real division is total: here $n(z) = 0$.
- Profit is zero, but the proposed wage identity says $1 = 1/2$.
- The separate counterexample compiles successfully.

This tests unrestricted algebra; it is not an admissible economic team.

Lean 6/6: what passed and what remains open

Status	Evidence
PASS	Support proofs, full build, and required fast check.
STOPPED	Semantic preflight: missing source statement map.
0 of 6	Full continuum propositions proved.

Matching, equilibrium, sorting, and source-level audits remain open.

What I learned

A checked proof is useful when its assumptions and scope are explicit.